Zadanie 1 – MyBlockchain

DISCLAIMER: Toto zadanie vychádza z knihy Bitcoin and Cryptocurrency Technologies od A. Narayana a z rovnomeného kurz učeného na Univerzite v Princetone.

Zadanie 1 je povinné! V zadaní 1 bude cieľom vytvoriť vlastnú blockchainovú sieť, ktorá bude zvládať základné dopyty, zaradí transakcie do blokov, jednotlivé bloky do reťaze blokov a dosiahne konsenzus medzi účastníkmi. Odporúčame rozdeliť si prácu na projekte do 3 fáz po jednotlivých týždňoch.

Fáza 1 – Jednoduchý „token“

V jednoduchom „tokene“ centrálna autorita FIIT prijíma transakcie od používateľov. Vašou úlohou bude implementovať logiku používanú FIIT na spracovávanie transakcií a vytvorenie distribuovanej účtovnej knihy (angl. ledger). FIIT zgrupuje transakcie do časových radov alebo blokov. V každom bloku dostane FIIT zoznam transakcií, tieto transakcie skontroluje a zverejní zoznam platných transakcií.

Pozor, že jedna transakcia sa môže odkazovať na inú v rovnakom bloku. Tiež sa môže stať, že medzi transakciami prijatými FIIT v jednom bloku sa nachádza viac ako jedna transakcia míňajúca rovnaký výstup (output). Toto by bolo samozrejme dvojité míňanie (angl. double-spending) a tým pádom transakcia neplatná. Znamená to, že transakcie nemôžu byť kontrolované izolovane; vaša výzva je, ako si vybrať podmnožinu transakcií, ktoré sú *spoločne* platné.

Máte k dispozícii súbor `Transaction` obsahujúci rovnomennú triedu, ktorá predstavuje transakciu jednoduchého „tokenu“ a má vnútorné triedy `Transaction.Output` a `Transaction.Input`. Transakčný výstup pozostáva z hodnoty a z verejného kľúča, na ktorý sa platí.

Transakčný vstup sa skladá z hash transakcie, ktorá obsahuje zodpovedajúci výstup, indexu tohto výstupu v danej transakcii (indexy sú jednoducho celé čísla začínajúce od 0) a digitálneho podpisu. Aby bol vstup platný, musí obsahovať podpis, ktorý musí byť platným podpisom nad aktuálnou transakciou s verejným kľúčom vo využívanom výstupe. Podpísané raw údaje sa získavajú metódou `getDataToSign(int index)`.

Transakcia pozostáva zo zoznamu vstupov, zoznamu výstupov a hash kompletnej transakcie (pozri `getTx()`). Trieda tiež obsahuje metódy na pridanie a odstránenie vstupu, pridanie výstupu, výpočet digestu na podpísanie/hash, pridanie podpisu na vstup (výpočet

podpisov vykonáva entita mimo triedy Transaction, ktorá pozná príslušné súkromné kľúče), a vypočítanie a uloženie hash transakcie po pridaní všetkých vstupov/výstupov/podpisov.

Taktiež máte k dispozícii súbor `UTXO`, ktorý predstavuje neminutý výstup transakcie. UTXO obsahuje hash transakcie, z ktorej pochádza, ako aj index v rámci tejto transakcie. Do UTXO boli zahrnuté funkcie `equals`, `hashCode` a `compareTo`, ktoré umožňujú testovanie rovnosti a porovnanie medzi dvoma UTXO na základe ich indexov a obsahu ich polí txHash.

Ďalej máte súbor `UTXOPool`, ktorý predstavuje aktuálnu množinu UTXO a obsahuje mapu z každého UTXO na jeho zodpovedajúci výstup transakcie. Tento súbor obsahuje konštruktory na vytvorenie nového prázdneho UTXOPool alebo kópie daného UTXOPool a metódy na pridanie a odstránenie UTXO z poolu, získanie výstupu zodpovedajúceho danému UTXO, skontrolovanie, či UTXO je v poole, a získanie zoznamu všetkých UTXO v poole.

Nakoniec vám je v tejto fáze poskytnutý súbor `rsa.jar` na používanie triedy `RSAKey`. Verejný kľúč vo výstupe transakcie a súkromný kľúč používaný na vytváranie podpisov na vstupoch transakcie sú oba reprezentované `RSAKey`, `RSAKeyPair` je pár verejný/súkromný kľúč. Ak vás to zaujíma, rozhrania API pre RSAKey a RSAKeyPair sú dostupné na stránke kurzu COS 432.

Vašou úlohou je vytvorenie súboru `HandleTxs.java`, ktorého základná štruktúra je k dispozícii a ktorý implementuje nasledujúce API:

```
public class HandleTxs {

    /**
     * Vytvorí verejný ledger, ktorého aktuálny UTXOPool (zbierka nevyčerpaných
     * transakčných výstupov) je {@code utxoPool}. Malo by to vytvoriť ochrannú kópiu
     * utxoPool pomocou konštruktora UTXOPool (UTXOPool uPool).
     */
    public HandleTxs(UTXOPool utxoPool);

    /**
     * @return true, ak
     * (1) sú všetky výstupy nárokované {@code tx} v aktuálnom UTXO pool,
     * (2) podpisy na každom vstupe {@code tx} sú platné,
     * (3) žiadne UTXO nie je nárokované viackrát,
     * (4) všetky výstupné hodnoty {@code tx}s sú nezáporné a
     * (5) súčet vstupných hodnôt {@code tx}s je väčší alebo rovný súčtu jej
     *     výstupných hodnôt; a false inak.
     */
    public boolean txIsValid(Transaction tx);

    /**
     * Spracováva každú epochu prijímaním neusporiadaného radu navrhovaných
     * transakcií, kontroluje správnosť každej transakcie, vracia pole vzájomne
     * platných prijatých transakcií a aktualizuje aktuálny UTXO pool podľa potreby.
     */
    public Transaction[] handler(Transaction[] possibleTxs);
}
```

Vaša implementácia `handler()` by mala vrátiť vzájomne platnú množinu transakcií maximálnej veľkosti (takú, ktorú nie je možné zväčšiť jednoduchým pridaním ďalších transakcií). Nemusíte počítať množinu maximálnej veľkosti všeobecne (takú, pre ktorú neexistuje väčšia vzájomne platná množina transakcií).

Na základe transakcií, ktoré sa rozhodla prijať, `handler()` by tiež mal aktualizovať svoj interný UTXOPool tak, aby odrážal aktuálnu množinu neminutých transakčných výstupov, aby budúce volania na `handler()` a `txIsValid()` boli schopné správne spracovať/overiť transakcie, ktoré si nárokujú výstupy z transakcií prijatých pri predchádzajúcom volaní na `handler()`.

Ďalej je potrebné vytvoriť druhý súbor s názvom `MaxFeeHandleTxs.java`, ktorého metóda `handler()` vyhľadá množinu transakcií s maximálnymi celkovými poplatkami za transakcie – tj. maximalizuje súčet všetkých transakcií v množine (súčet vstupných hodnôt - súčet výstupných hodnôt). Nájdenie takejto množiny transakcií je NP ťažké, a preto nájdite iba heuristiku, ktorá by mohla zvýšiť celkové transakčné poplatky.

Fáza 2 – Dôvera a konsenzus

V tejto časti navrhnete a implementujete algoritmus distribuovaného konsenzu s grafom vzťahov „dôvery“ medzi uzlami. Toto je alternatívna metóda odolávania sybil útokom a dosiahnutia konsenzu; má výhodu v tom, že „neplytváte“ elektrickou energiou, ako to robí proof of work.

Uzly v sieti sú buď dôveryhodné, alebo podvodné. Napíšete triedu `TrustedNode` (implementuje poskytované rozhranie `Node`), ktorá definuje správanie každého z dôveryhodných uzlov. Sieť je orientovaný náhodný graf, kde každá hrana predstavuje vzťah dôvery. Napríklad, ak existuje hrana A → B, znamená to, že uzol B počúva transakcie vysielané uzlom A. Hovoríme, že B je nasledovník A a A je nasledovaný B.

Každý uzol by mal uspieť pri dosahovaní konsenzu so sieťou, v ktorej sú jeho peermi ďalšie uzly používajúce rovnaký kód. Váš algoritmus by mal byť navrhnutý tak, aby sa sieť uzlov prijímajúca rôzne skupiny transakcií mohla dohodnúť na skupine, ktorú akceptujú. Máte k dispozícii triedu `Simulation`, ktorá generuje náhodný graf dôvery. Nastavte si počet kôl, počas ktorých budú vaše uzly v každom kole vysielať svoj návrh svojim nasledovníkom a na konci kola by mali dosiahnuť konsenzus o tom, na ktorých transakciách by sa mali dohodnúť.

Každý uzol dostane svoj zoznam sledovaných osôb prostredníctvom boolovského poľa, ktorého indexy zodpovedajú uzlom v grafe. Hodnota „true“ v indexe i označuje, že uzol i je sledovaný, inak „false.“ Tento uzol tiež získa zoznam transakcií (zoznam ponúk), ktoré môže

vysielať svojim sledujúcim. Za vytvorenie počiatočných transakcií/zoznamu návrhov nebudete zodpovedať. Predpokladajme, že všetky transakcie sú platné a že neplatné transakcie nemožno vytvoriť. Poskytnutá trieda Node má nasledujúce API:

```
public interface Node {

    // POZNÁMKA: Node je rozhranie a nemá konštruktor.
    // Vaša trieda TrustedNode.java však vyžaduje 4-argumentový
    // konštruktor, ako je definované v Simulation.java
    // Tento konštruktor dáva vášmu uzlu informácie o simulácii
    // vrátane počtu kôl, pre ktoré bude bežať.

    /**
     * {@code followees [i]} je pravda iba ak tento uzol nasleduje uzol {@code i}
     */
    void followeesSet(boolean[] followees);

    /** inicializovať navrhovaný zoznam transakcií */
    void pendingTransactionSet(ArrayList<Transaction> pendingTransactions);

    /**
     * @return návrhy, ktoré pošlem mojim nasledovníkom. Pamätajme: Po finálovom
     *         kole sa správanie {@code followersSend} zmení a malo by vrátiť
     *         transakcie, pri ktorých bol dosiahnutý konsenzus.
     */
    Set<Transaction> followersSend();

    /** príjmy kandidátov z iných uzlov */
    void followeesReceive(ArrayList<Integer[]> candidates);
}
```

Pri testovaní sa môžu uzly, na ktorých beží váš kód, stretnúť s množstvom (až 45%) škodlivých uzlov, ktoré nespolupracujú s vaším konsenzuálnym algoritmom. Uzly vášho návrhu by mali byť schopné vydržať čo najviac škodlivých uzlov a stále dosiahnuť konsenzus. Škodlivé uzly môžu mať svojvoľné správanie. Napríklad, škodlivý uzol môže:

- byť funkčne mŕtvy a nikdy v skutočnosti nevysielať žiadne transakcie,
- neustále vysiela svoju vlastnú skupinu transakcií a nikdy neprijíma transakcie, ktoré sú mu dané,
- meniť správanie medzi kolami, aby sa zabránilo detekcii.

K dispozícii máte nasledujúce súbory:

`Node.java`	Základné rozhranie pre vašu triedu TrustedNode.
`TrustedNode.java`	Kostra triedy TrustedNode. Váš kód by ste mali implementovať na základe šablóny, ktorú tento súbor poskytuje.
`ByzantineNode.java`	Veľmi jednoduchý príklad škodlivého uzla.

`Simulation.java`	Základný generátor grafov, ktorý môžete použiť na spustenie vlastných simulácií s rôznymi parametrami grafu a na otestovanie triedy TrustedNode.
`Transaction.java`	Trieda Transaction, pričom transakcia je iba obalom okolo jedinečného identifikátora (t. j. platnosť a sémantika transakcií sú pre toto priradenie irelevantné).

Graf uzlov bude mať nasledujúce parametre:

- pravdepodobnosť párovej konektivity náhodného grafu: napr. {.1, .2, .3},
- pravdepodobnosť, že uzol bude nastavený ako podvodný: napr. {.15, .30, .45},
- pravdepodobnosť, že bude oznámená každá z počiatočných platných transakcií: napr. {.01, .05, .10},
- počet kôl v simulácii napr. {10, 20}.

Zamerajte sa na vývoj robustnej triedy TrustedNode, ktorá bude pracovať so všetkými kombináciami parametrov grafu. Na konci každého kola váš uzol uvidí zoznam transakcií, ktoré mu boli vysielané.

Každý test sa meria na základe:

- Či dosiahne 100% konsenzus. To znamená, že všetky TrustedNode musia mať na výstupe rovnaké zoznamy. V tomto prípade neexistuje nič také ako 85% úspešnosť: konsenzus sa bude považovať za binárny. Všetky dôveryhodné uzly musia vrátiť rovnaké zoznamy, aby bol test úspešný.
- Veľkosť súboru, na ktorom sa dosahuje konsenzus. Mali by ste sa usilovať o dosiahnutie konsenzu čo najviac transakcií.
- Čas vykonania, ktorý by mal byť rozumný (ak kód trvá príliš dlho, na hodnotení sa to odrazí).

Uvedomme si, že test môžete mať úspešný aj jednoduchým vrátením prázdnej množiny, ale toto nie je dobrý konsenzuálny algoritmus. Mali by ste sa snažiť o to, aby bol konsenzuálny súbor transakcií čo najväčší.

Niektoré rady:

- Váš uzol nebude poznať topológiu siete a mal by sa snažiť pracovať všeobecne. To znamená, že si musí uvedomiť, ako rôzna topológia môže mať vplyv na to, ako chceme zahrnúť transakcie podľa obrazu konsenzu.

- Váš kód TrustedNode môže predpokladať, že všetky transakcie, ktoré vidí, sú platné – simulácia pošle iba platné transakcie (počiatočné aj medzi kolami) a iba simulačný kód má schopnosť vytvárať platné transakcie.
- Ignorujte hraničné prípady, ktoré sa vyskytujú s extrémne nízkou pravdepodobnosťou, napríklad keď sa dôveryhodný uzol spáruje iba so škodlivými uzlami. Testovacie prípady by takéto scenáre nemali mať.

Fáza 3 – Blockchain

V tejto fáze budete implementovať uzol, ktorý je súčasťou distribuovaného konsenzuálneho protokolu založeného na blokových reťazcoch. Váš kód bude konkrétne prijímať prichádzajúce transakcie a bloky a udržiavať aktualizovaný blockchain.

Poskytnuté súbory:

`Block.java`	Ukladá štruktúru dát bloku.
`HandleBlocks.java`	Používa BlockChain.java na spracovanie novo prijatého bloku, vytvorenie nového bloku alebo na spracovanie novo prijatej transakcie.
`ByteArrayWrapper.java`	Súbor obslužného programu, ktorý vytvára wrapper pre bajtové polia, takže ich možno použiť ako kľúč v hashovacích funkciách.
`Transaction.java`	Podobné ako v prípade Transaction.java vo Fáze 1, s výnimkou zavedenia funkcií na vytvorenie transakcie coinbase. Prezrite si konštruktor Block.java a uvidíte, ako sa vytvára coinbase.
`TransactionPool.java`	Implementuje pool transakcií, ktorý sa vyžaduje pri vytváraní nového bloku.
`UTXO.java`	Z fázy 1.
`UTXOPool.java`	Z fázy 1.

Vytvorte verejnú funkciu `UTXOPoolGet()` v súbore `HandleTxs.java`, ktorý ste vytvorili vo fáze 1 a skopírujte súbor do svojho projektu pre fázu 3.

```java
public class HandleTxs {

    /**
     * Vytvorí verejný ledger, ktorého aktuálny UTXOPool (zbierka nevyčerpaných
     * transakčných výstupov) je {@code utxoPool}. Malo by to vytvoriť ochrannú kópiu
     * utxoPool pomocou konštruktora UTXOPool (UTXOPool uPool).
     */
    public HandleTxs(UTXOPool utxoPool);

    /**
     * @return aktuálny UTXO pool.
     * Ak nenájde žiadny aktuálny UTXO pool, tak vráti prázdny (nie nulový) objekt {@code
     * UTXOPool}.
     */
    public UTXOPool UTXOPoolGet();

    /**
     * @return true, ak
     * (1) sú všetky výstupy nárokované {@code tx} v aktuálnom UTXO pool,
     * (2) podpisy na každom vstupe {@code tx} sú platné,
     * (3) žiadne UTXO nie je nárokované viackrát,
     * (4) všetky výstupné hodnoty {@code tx}s sú nezáporné a
     * (5) súčet vstupných hodnôt {@code tx}s je väčší alebo rovný súčtu jej
     *     výstupných hodnôt; a false inak.
     */
    public boolean txIsValid(Transaction tx);

    /**
     * Spracováva každú epochu prijímaním neusporiadaného radu navrhovaných
     * transakcií, kontroluje správnosť každej transakcie, vracia pole vzájomne
     * platných prijatých transakcií a aktualizuje aktuálny UTXO pool podľa potreby.
     */
    public Transaction[] handler(Transaction[] possibleTxs);
}
```

Trieda Blockchain je zodpovedná za udržiavanie reťaze blokov. Pretože celý blockchain môže mať obrovskú veľkosť, mali by ste si ukladať iba pár posledných blokov. Presné číslo, ktoré sa má uložiť, závisí od vášho návrhu, teda pokiaľ budete môcť implementovať všetky API funkcie popísané v tomto dokumente.

Nakoľko môže byť viac forkov, bloky tvoria skôr strom ako zoznam. Váš návrh by to mal brať do úvahy. Musíte udržiavať UTXO pool zodpovedajúci každému bloku, nad ktorým by sa mohol vytvoriť nový blok.

Súbor, ktorý sa má upraviť:

`Blockchain.java`

```java
// Blockchain by mal na naplnenie funkcií udržiavať iba obmedzené množstvo uzlov
// Nemali by ste mať všetky bloky pridané do blockchainu v pamäti
// pretože by to spôsobilo pretečenie pamäte.

public class Blockchain {
    public static final int CUT_OFF_AGE = 12;

    /**
     * vytvor prázdny blockchain iba s Genesis blokom. Predpokladajme, že
     * {@code genesisBlock} je platný blok
     */
    public BlockChain(Block genesisBlock) {
        // IMPLEMENTOVAŤ
    }

    /** Získaj maximum height blok */
    public Block getBlockAtMaxHeight() {
        // IMPLEMENTOVAŤ
    }

    /** Získaj UTXOPool na ťaženie a nový blok na vrchu max height blok */
    public UTXOPool getUTXOPoolAtMaxHeight() {
        // IMPLEMENTOVAŤ
    }

    /** Získaj pool transakcií na vyťaženie nového bloku */
    public TransactionPool getTransactionPool() {
        // IMPLEMENTOVAŤ
    }

    /**
     * Pridaj {@code block} do blockchainu, ak je platný. Kvôli platnosti by mali
     * byť všetky transakcie platné a blok by mal byť na
     * {@code height > (maxHeight - CUT_OFF_AGE)}.
     *
     * Môžete napríklad vyskúšať vytvoriť nový blok nad blokom Genesis (výška bloku
     * 2), ak height blockchainu je {@code <=
     * CUT_OFF_AGE + 1}. Len čo {@code height > CUT_OFF_AGE + 1}, nemôžete vytvoriť
     * nový blok vo výške 2.
     *
     * @return true, ak je blok úspešne pridaný
     */
    public boolean blockAdd(Block block) {
        // IMPLEMENTOVAŤ
    }

    /** Pridaj transakciu do transakčného poolu */
    public void transactionAdd(Transaction tx) {
        // IMPLEMENTOVAŤ
    }
```

Predpoklady a rady:

- Nový Genesis blok sa nebude ťažiť. Ak dostanete blok, ktorý o sebe tvrdí, že je blokom genesis (rodič je nulový hash), vo funkcii `blockAdd(Block b)`, môžete vrátiť hodnotu false.
- Ak je v rovnakej výške viac blokov, vráťte najstarší blok vo funkcii `getBlockAtMaxHeight()`.
- Pre jednoduchosť predpokladajme, že je k dispozícii coinbase transakcia bloku, ktorá sa môže minúť v nasledujúcom bloku ťaženom nad ňou (To je v rozpore so skutočným Bitcoin protokolom, keď pred jeho uplatnením existuje finalita niekoľko potvrdení).
- Udržiavajte iba jeden globálny pool transakcií pre blockchain a pri prijímaní transakcií doň neustále pridávajte transakcie a odstráňte z neho transakcie, ak je prijatý alebo vytvorený nový blok. Je v poriadku, ak dôjde k zrušeniu niektorých transakcií počas reorganizácie blockchainu, tj. keď sa bočná vetva stane novou najdlhšou vetvou. Konkrétne transakcie prítomné v pôvodnej hlavnej vetve (a teda odstránené z poolu transakcií), ktoré však chýbajú v bočnej vetve, sa môžu stratiť. Napríklad v reťazci A je prijatý blok obsahujúci transakciu Tx1. Odstránime Tx1 z poolu transakcií. Teraz predpokladajme, že reťazec B je dlhší ako reťazec A. Nevracajte Tx1 späť do poolu, aj keď v ideálnom prípade by sa mala vrátiť späť. Toto zjednoduší vašu prácu, ako aj naše testovanie. (Mineri nie sú zodpovední za zahrnutie transakcií do blokov. Ak dôjde k strate transakcie, je zodpovednosťou vlastníka transakcie znovu ju poslať do siete.)
- Hodnota coinbase je udržiavaná na konštantnej hodnote 3.125 bitcoinov, zatiaľ čo v skutočnosti sa znižuje na polovicu zhruba každé 4 roky.
- Pri kontrole platnosti novo prijatého bloku stačí iba skontrolovať, či sú transakcie z platnej množiny. Táto množina nemusí predstavovať maximálny možný počet transakcií. Tiež nemusíte robiť žiadne kontroly funkčnosti alebo či blok obsahuje nejaké množstvo 0 (nerobíme žiadny proof of work).

Extra body: 3 body naviac získajú tí, ktorí umožnia v blockchaine používať multisig transakcie a zároveň ich overenie v štandardnom procese ťažby nových blokov.

Odovzdanie

Dokumentáciu a zdrojový kód implementácie študent odovzdáva v elektronickom tvare do AISu do **05.04.2026 23:59**. **Odovzdanie aspoň nejakej časti zadania je povinné!** Aj na toto zadanie je možné využiť Late days. Všetky kódy prejdú kontrolou originality, v prípade vysokej percentuálnej zhody bude študentovi začaté disciplinárne konanie.

Každý program bude testovaný automatizovane pomocou unit testov. Podmienkou je aj aktívne odprezentovanie programu študentom na cvičení podľa harmonogramu.

Súčasťou riešenia je aj dokumentácia, ktorá musí obsahovať najmä:

a) formálne náležitosti ako sú titulná strana, číslovanie strán;
b) vysvetlené jednotlivé fázy a ako sa s nimi študent popasoval;
c) používateľská príručka;
d) voľbu implementačného prostredia a prípadné napísané testy;
e) iné záležitosti, ktorými sa študent chce pochváliť;
f) záver a v ňom zhodnotenie vlastného príspevku a čo som sa naučil/a.

Hodnotenie (bez minima)

Celé riešenie - max. 20 bodov, z toho:

- max. 9 bodov za riešenie úlohy vo fáze 1; 18 unit testov každý za 0,5b
- max. 2 body za riešenie úlohy vo fáze 2; 4 unit testy každý za 0,5b
- max. 6,75 bodu za riešenie úlohy vo fáze 3; 27 unit testov každý za 0,25b
- max. 2,25 bodu za výslednú dokumentáciu.

Bonusy – dajú sa získať naviac k bodom za zadanie

- max. 3 body za MultiSig transakcie vo fáze 3; 9 unit testov každý za 0,33b
- max. 2 body za GUI a vizualizačný nástroj, ktorý uľahčí interakciu so zadaním
- max. 5 bodov za prácu v inom jazyku (napr. Python) a prerobenie všetkých zdrojových kódov vrátane vlastných unit testov na Python 3.

V prípade požiadavky vieme dodať starter code aj v jazyku Rust, GO.

AI usage

- využitie AI nástrojov je vo všeobecnosti tolerované, ak to explicitne uvediete a priznáte,
- v dokumentácii treba presne popísať, kde a na aký účel a aké konkrétne AI bolo použité,
- v zdrojových kódoch vo forme komentáru treba uviesť, aké konkrétne AI bolo použité a do akej miery.